

Phase 5: Apex Programming

Step 1: Classes & Objects – Dealer Recommendation Logic

Goal: Create a reusable Apex class to recommend the nearest dealer for a given vehicle.

Steps:

1. **Go to Setup → Apex Classes → New**
2. **Example Apex Class: DealerRecommendation**

```
public with sharing class DealerRecommendation {
```

```
    // Method to fetch nearest dealer based on customer location
```

```
    public static Dealer__c getNearestDealer(Account customer, Vehicle__c vehicle) {
```

```
        // Query dealers with the vehicle in stock
```

```
        List<Dealer__c> dealers = [
```

```
            SELECT Id, Name, Location__Latitude__s, Location__Longitude__s, Stock__c
```

```
            FROM Dealer__c
```

```
            WHERE Stock__c >= :vehicle.Quantity__c
```

```
        ];
```

```
        if (dealers.isEmpty()) {
```

```
            return null; // No dealers available
```

```
        }
```

```
        Dealer__c nearestDealer;
```

```
        Double minDistance = Double.MAX_VALUE;
```

```

for (Dealer__c d : dealers) {

    Double distance = GeoUtils.calculateDistance(

        customer.BillingLatitude,

        customer.BillingLongitude,

        d.Location__Latitude__s,

        d.Location__Longitude__s

    );

    if (distance < minDistance) {

        minDistance = distance;

        nearestDealer = d;

    }

}

return nearestDealer;

}
}

```

✅ **Tip:** Create a GeoUtils helper class for distance calculations if needed.

Step 2: Apex Triggers – Prevent Orders for Out-of-Stock Vehicles

Goal: Stop orders from being created if the vehicle stock is insufficient.

1. **Go to Object Manager → Order → Triggers → New**
2. **Trigger Example: Prevent Out-of-Stock Orders**

```

trigger OrderTrigger on Order__c (before insert, before update) {

    for (Order__c o : Trigger.new) {

```

```

Vehicle__c vehicle = [SELECT Stock__c FROM Vehicle__c WHERE Id = :o.Vehicle__c LIMIT 1];

if (vehicle.Stock__c < o.Quantity__c) {

    o.addError('Cannot create order: Vehicle stock is unavailable.');
```

Step 3: SOQL & SOSL – Fetch Dealers & Stock Status

- Use **SOQL** to fetch dealer and stock info (as in class above).
- Use **SOSL** for searching by dealer name or location:

```
List<List<SObject>> searchResults = [FIND 'BestDealer*' IN ALL FIELDS RETURNING Dealer__c(Id, Name)];
```

Step 4: Batch Apex – Scheduled Updates for Bulk Orders

Goal: Handle large datasets of bulk orders for stock updates.

1. Create Apex Class implementing Database.Batchable<SObject>

```

global class BulkOrderStockUpdate implements Database.Batchable<SObject> {

    global Database.QueryLocator start(Database.BatchableContext BC) {

        return Database.getQueryLocator([SELECT Id, Vehicle__c, Quantity__c FROM Order__c WHERE
        Status__c = 'Pending']);

    }

    global void execute(Database.BatchableContext BC, List<Order__c> orders) {

        List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

        for (Order__c o : orders) {
```

```

        Vehicle__c v = [SELECT Id, Stock__c FROM Vehicle__c WHERE Id = :o.Vehicle__c LIMIT 1];

        v.Stock__c -= o.Quantity__c;

        vehiclesToUpdate.add(v);
    }

    update vehiclesToUpdate;
}

global void finish(Database.BatchableContext BC) {

    System.debug('Bulk stock update completed.');
```

Schedule this using System.schedule() or Salesforce UI.

Step 5: Queueable & Scheduled Apex – Automate Stock Validation

Queueable Example:

```

public class StockValidationJob implements Queueable {

    public void execute(QueueableContext context) {

        List<Order__c> orders = [SELECT Id, Vehicle__c FROM Order__c WHERE Status__c = 'Pending'];

        for (Order__c o : orders) {

            // Reuse dealer recommendation logic

            Dealer__c dealer = DealerRecommendation.getNearestDealer(o.Account__r, o.Vehicle__r);

            if (dealer == null) {

                o.addError('No dealer with stock available.');
```

```

    }

    update orders;

}

}

```

Scheduled Apex Example:

```

global class ScheduledStockValidation implements Schedulable {

    global void execute(SchedulableContext sc) {

        System.enqueueJob(new StockValidationJob());

    }

}

```

Step 6: Future Methods – Call External APIs

```

public class ExternalStockSync {

    @future(callout=true)

    public static void syncStock(Id vehicleId) {

        Vehicle__c v = [SELECT Id, External_API_URL__c FROM Vehicle__c WHERE Id = :vehicleId];

        HttpRequest req = new HttpRequest();

        req.setEndpoint(v.External_API_URL__c);

        req.setMethod('GET');


        Http http = new Http();

        HttpResponse res = http.send(req);


        // Process response and update stock

        v.Stock__c = Integer.valueOf(res.getBody());
    }
}

```

```
        update v;
    }
}
```

Step 7: Exception Handling – Graceful Errors

```
try {
    ExternalStockSync.syncStock(vehicleId);
} catch (Exception e) {
    System.debug('Error syncing stock: ' + e.getMessage());
}
```

Step 8: Test Classes – Ensure Automation Works

```
@isTest

public class DealerRecommendationTest {

    @testSetup
    static void setupData() {
        Vehicle__c v = new Vehicle__c(Name='Car A', Stock__c=10);
        insert v;

        Dealer__c d = new Dealer__c(Name='Dealer 1', Stock__c=5);
        insert d;
    }

    static testMethod void testNearestDealer() {
```

```
Account acc = new Account(Name='Customer', BillingLatitude=12.97, BillingLongitude=77.59);  
insert acc;
```

```
Vehicle__c v = [SELECT Id FROM Vehicle__c LIMIT 1];
```

```
Dealer__c dealer = DealerRecommendation.getNearestDealer(acc, v);
```

```
System.assertNotEquals(null, dealer, 'Dealer should not be null');
```

```
}
```

```
}
```